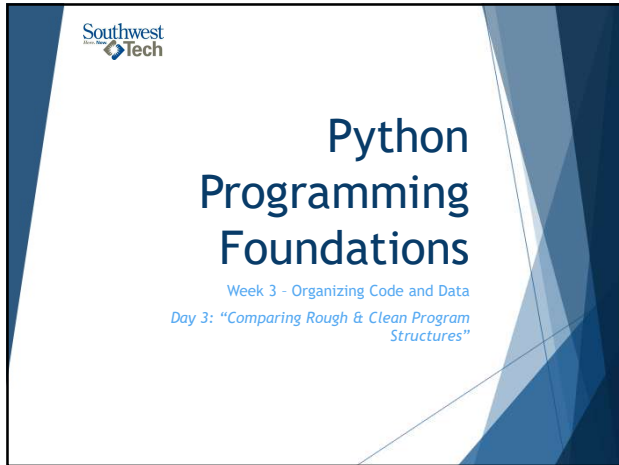
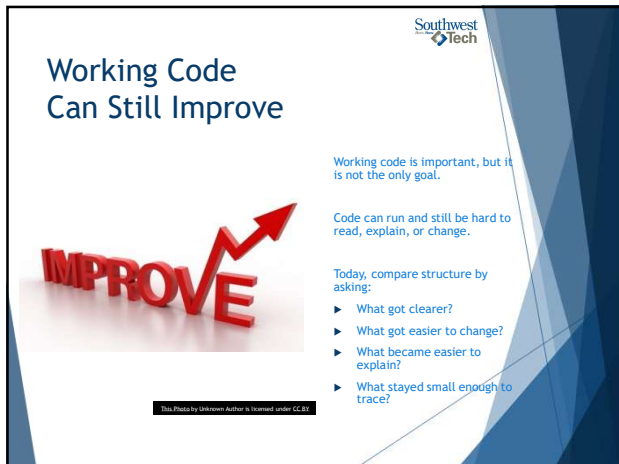
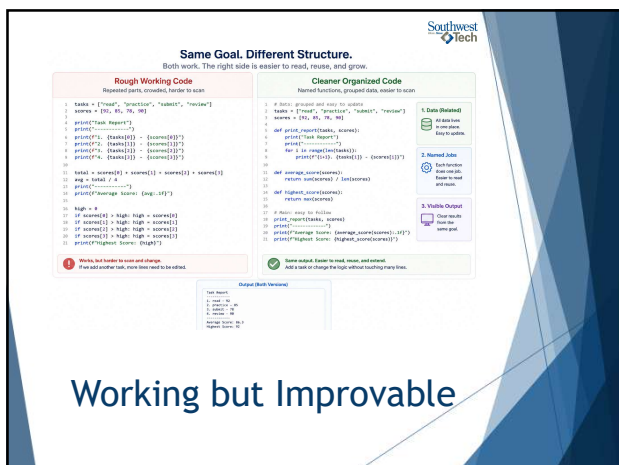




1



2



3



Functions Plus Data Structures

Functions organize logic.

Lists and dictionaries organize information.

Cleaner programs often use both:

- ▶ named jobs
- ▶ related data grouped together
- ▶ output that is easier to explain

4



Functions Work With Data Structures

Named jobs + related data = visible output

1. Named Jobs (Functions)

Functions are reusable steps that do a job.

- `get_tasks()`
Get the list of tasks
- `add_task(task)`
Add a task to the list
- `show_tasks(tasks)`
Print each task

2. Related Data (Data Structures)

Data is stored in structures that functions can use.

```
tasks (list)
• read
• practice
• submit

new_task (string)
"review"
```

3. Visible Output (Results)

The results are shown so people can see them.

```
Task List
1. read
2. practice
3. submit
4. review
```

How It Works (Example)

```
get_tasks() → show_tasks(tasks_list) → add_task("review") → tasks_list updated → show_tasks(tasks_list) → visible output on the screen
```

Key Idea: Functions (named jobs) work with data (related data) to produce results we can see (visible output).

Functions Plus Data

5

What Counts as Success Today

Working baseline ->
compare structure ->
revise one meaningful thing ->
explain the improvement.

↓

A useful revision should make the program:

clearer	easier to test	easier to change	easier to explain
---------	----------------	------------------	-------------------

6

Southwest
Tech

Revise in Small, Meaningful Steps

A simple 4-step flow you can repeat.

1 Working Baseline

Start with something that already works.

Example

```
def task():
    read()
    practice()
    submit()
    print('done')
```

→ Prints each task

2 Compare Structure

Look at your data and code structure.

Check

- Is the structure a good match for the job?
- Is the code easy to read and reuse?
- Is anything repeated unnecessarily?

3 Revise One Meaningful Thing

Make one clear change that improves the code or the data.

Example Change

Before:
Repeated prints
↓
After:
Loop to print each task
Small change, big impact.

4 Explain Improvement

Say what is better and why it matters.

My Improvement

I used a loop to print each task.
Now the code is shorter, easier to read, and easier to update.

Keep It Going
Repeat these four steps often. Small improvements add up to strong, clear programs.

Today's Success

7

Southwest
Tech

What We Will Use Today

- ▶ rough working code
- ▶ cleaner organized code
- ▶ function responsibility
- ▶ list/dictionary choice
- ▶ comparison questions
- ▶ small revision

The goal is better structure, not more features.

8

Southwest
Tech

Revision Toolbox

Simple tools to make your code and data better, step by step.

1. Rough Baseline

Get something working. Don't worry about perfect.

Example

```
def task():
    print('Task Report')
    print('Task 1')
    print('Task 2')
    print('Task 3')
```

2. Cleaner Version

Organize code and data so it's easier to read.

Example

```
tasks = ['read', 'write', 'delete']
for i, task in enumerate(tasks, 1):
    print(f'{i}: {task}')
```

3. Function Responsibility

Each function does one clear job.

Example

```
def get_data():
    return data
def add_task(task):
    tasks.append(task)
def display_tasks():
    for task in tasks:
        print(task)
```

4. Data Choice

Pick the structure that fits the job.

List (ordered)

- keep order
- look through
- use positions

Dictionary (unordered)

- easy to find
- look up values
- clearer names

5. Comparison Questions

Ask these before you make changes.

- Does the data structure fit the job?
- Is the code easy to read and reuse?
- Is anything repeated unnecessarily?
- Will this make future changes easier?

6. Small Revision

Make one meaningful change at a time.

Examples

- Replace repeated prints with a loop
- Move repeated code into a function
- Reorganize something to be clearer
- Choose a better data structure

Use these tools often.
Small steps lead to big improvements.

Remember: Clear code = good data choices + easy to read, reuse, and grow.

Toolbox

9

Southwest
Tech

What We Will “Skip” For Now

- ▶ Broad redesign
- ▶ Class-based architecture
- ▶ Large AI rewrites
- ▶ Project Restructuring

Today's revision should be small enough to explain:

- ▶ one function improvement
- ▶ one data-structure improvement
- ▶ one clearer name
- ▶ one removed repetition

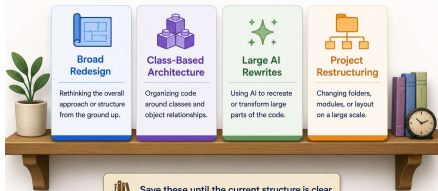


10

Southwest
Tech

Parked for Later

Powerful ideas. Best used when the basics are solid.



Save these until the current structure is clear.

Parked For Later

11

Southwest
Tech

Correct Is Not The Same As Clear

Correct code produces the expected result.
Clear code helps a human understand how the result happens.

Both matter because:

- ▶ programs need to run
- ▶ people need to read them
- ▶ future changes need to be possible



12

Southwest
Tech

Two Sides of Good Code

Great code is both correct and clear.

Correct
The code produces the expected output.

Expected Output

```
Task Report
-----
1. read      - 92
2. practice  - 85
3. submit    - 78
4. review    - 98
-----
Average Score: 86.3
Highest Score: 92
```

- ☒ Runs without errors
- ☒ Matches the expected results
- ☒ Solves the problem correctly

Clear
The code is easy to read, understand, and explain.

Readable Structure

```
def average_score(scores):
    return sum(scores) / len(scores)

def highest_score(scores):
    return max(scores)

def print_report(tasks, scores):
    for i, (task, score) in enumerate(zip(tasks, scores)):
        print(f"{i+1}: {task} - {score}")

# Example usage
tasks = ["read", "practice", "submit", "review"]
scores = [92, 85, 78, 98]
print_report(tasks, scores)
```

Clear Explanation

- Functions do one job each
- Data is grouped and easy to follow
- Easy to change, test, and reuse

- ☒ Easy to read and follow
- ☒ Easy to explain to someone else
- ☒ Easy to change and build on

Both matter equally.
 Code should be correct in what it does and clear in how it does it.

Correct vs. Clear

13

Southwest
Tech

Cleaner Structure Is Easier To Explain

Cleaner structure usually makes the program easier to talk through - Look for:

- ▶ clear function names
- ▶ related data grouped together
- ▶ less repeated logic
- ▶ output that matches the program's purpose

If you can explain it better, the structure probably improved.

14

Southwest
Tech

Clear Functions + Grouped Data = Understanding

Good names. Good groups. Easy to explain.

Clear Function Names
Each function does one clear job.

```
get_tasks()
    Get the list of tasks
add_task(task)
    Add one task
calculate_score(tasks)
    Calculate total score
show_summary(tasks, score)
    Display the results
```

Grouped Data
Related data lives together.

```
tasks (list)
    "read", "practice", "submit", "review"

scores (list)
    92, 85, 78, 98

settings (dict)
    "setting": 70
    "show_tasks": True
```

Works Together
Data flows through clear steps.

```
get_tasks()
    ↓
add_task(task)
    ↓
calculate_score(tasks)
    ↓
show_summary(tasks, score)
    ↓
show_tasks
```

I can explain this.

Clear names
Make the purpose obvious.

Grouped data
Keep related things together.

Easy to explain
You don't often understand the flow.

Explainable Structure

15



Comparison Questions

Use comparison questions instead of vague opinions.

Ask:

- ▶ What changed?
- ▶ Why does it help?
- ▶ What became easier?
- ▶ What stayed the same?
- ▶ What would I change next?

16



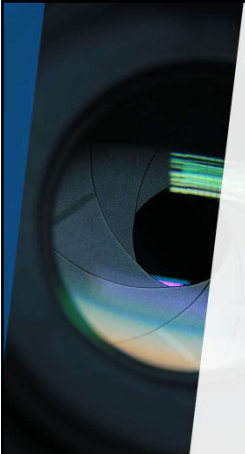
Quick Review Checklist

Use these five questions to review any revision.

1	↔	What changed? Identify the specific change you made.	
2	💡	Why does it help? Explain how this change improves the code or data.	
3	👍	What became easier? Describe what is easier to do now.	
4	⊖	What stayed the same? List what did not change (and why that's okay).	
5	📈	What would I change next? Note one small improvement to try next.	

Comparison Questions

17



Demo #1

"Rough vs Organized"

18



What to Watch For

Watch both versions run.

Then compare:

- ▶ repeated code
- ▶ function responsibility
- ▶ name clarity
- ▶ how easy each version is to change

19



What Changed?

A useful comparison names specific changes.

Examples:

- ▶ repeated output formatting moved into a function
- ▶ status logic received a clear name
- ▶ task data became easier to reuse
- ▶ one change affects multiple outputs

Specific changes are easier to justify than general impressions.

20



Before and After: Same Result, Better Structure

Same output. Less repetition. Clearer intent.

Rough Version
Repeated formatting for each task

```

Task 1
# build the header
print("-----")
print("1. Name")
print("Status:", "Done")
print("-----")

Task 2
# build the header
print("-----")
print("2. Project")
print("Status:", "Done")
print("-----")

Task 3
# build the header
print("-----")
print("3. Report")
print("Status:", "Done")
print("-----")

```

Issues: Harder to scan, harder to change, easy to introduce inconsistencies.

Organized Version
Use a function to format each task

```

format_task
# format task and display one task
# format_task(customer, title, status)
print(f"number: {task_id}")
print(f"Status: {status}")
print("-----")

Main Flow
format_task(1, "Name", "Done")
format_task(2, "Project", "Done")
format_task(3, "Report", "Done")

Output (same as before)
1. Name
Status: Done
2. Project
Status: Done
3. Report
Status: Done

```

Job Named: This job has a name and one clear job.

Repetition Reduced: The formatting happens once (for gets reused).

Same Result: The output is unchanged.

Benefits: Easier to read, easier to change, less work for errors.

Demo 1 Comparison

21



What Got Easier?

An improvement should make something easier.

Ask whether the revision made it easier to:

- ▶ read
- ▶ test
- ▶ change
- ▶ reuse
- ▶ explain

If nothing got easier, the revision may not be useful yet.

22



Good Improvements Make Work Easier

Any change is worth it if it makes at least one of these easier.

 read Understand the code quickly without confusion.	 test Check behavior and catch issues with confidence.	 change Make updates without breaking other parts.	 reuse Use the code or logic again in other places.	 explain Describe how it works to others clearly.
--	--	--	---	---

★ **Aim for progress.**
If your change makes at least one of these easier, you're on the right track.

What Got Easier

23



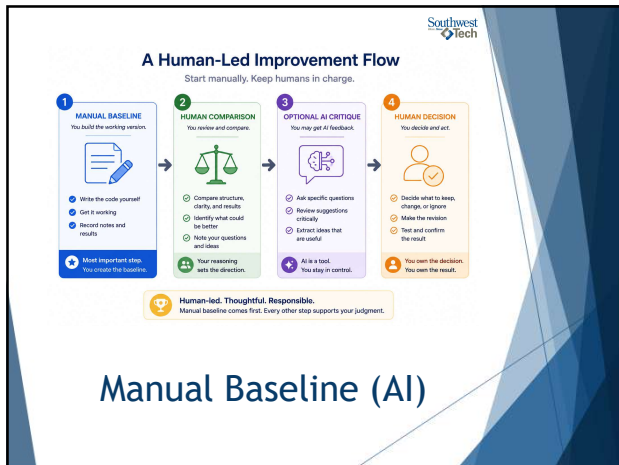
Manual Baseline Comes First (AI use)

AI critique only helps learning after you have your own baseline.

Use this order:

1. Build or revise manually.
2. Run and inspect your version.
3. Compare structure yourself.
4. Use AI only if permitted.
5. Decide what to keep, change, or reject.

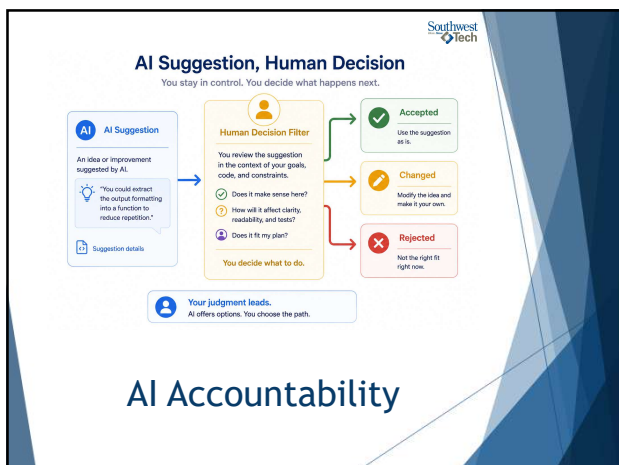
24



25



26



27



Finish Assignment #4

Check that your function structure is explainable.

Your program should have:

- ▶ at least two functions
- ▶ meaningful function names
- ▶ correct function calls
- ▶ one clear responsibility per function
- ▶ explanation of how the functions work together

28



Small Functions. Clear Responsibilities.

Each function does one job. Each job has a name.

calculate_total
(items)

Responsibility
Add up item prices and return the total.

Used by: show_summary

format_currency
(amount)

Responsibility
Convert a number to a currency string for display.

Used by: show_summary

show_summary
(items)

Responsibility
Get the total, format it, and display a summary.

Used by: (main program)

Function Review Checklist

- Names: Are the function names clear and specific?
- Calls: Are the functions called in the right order?
- Responsibilities: Does each function do one well-defined job?
- Explanation: Can I explain what each function does in one sentence?

Clear names. Single purpose. Easy to explain. That's the goal.

Finish Assignment #4

29



Finish Assignment #5

Check that your data structure is useful.

Your program should have:

- ▶ list or dictionary data
- ▶ a lookup, retrieval, or loop
- ▶ visible output
- ▶ a structure choice you can explain
- ▶ code that stays small enough to trace

30

Southwest
Tech

Data Mini-App: From Structure to Output

Good structure makes the output easy to produce and easy to explain.

1. Data (Structure Choice)

We'll use a list of dictionaries to store student scores.

```
students = [
    { "name": "Megan", "score": 92 },
    { "name": "Jordan", "score": 88 },
    { "name": "Lucia", "score": 78 }
]
```

Why this structure?
Each student has two related pieces of data (name and score). A list keeps the students in order.

2. Mini-App Logic

Loop through the list and build the output lines.

- Do through each student
- Create a line with name and score
- Collect the lines

3. Visible Output

Clear, readable result from our data.

```
Student Scores
-----
Megan    92
Jordan   88
Lucia    78
```

Structure Choice Explained

A list of dictionaries fits this problem because:

- Each student has multiple related values (name and score).
- We need to keep the students in order.
- It's easy to loop through and format for display.

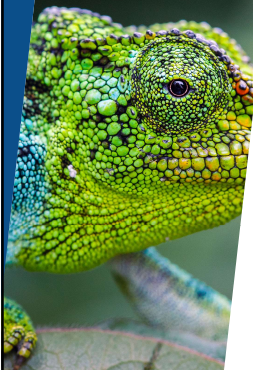
Good data structure → Simple logic → Clear output → Easy explanation

Finish Assignment #5

31

Southwest
Tech

Revision Evidence



It should explain what improved.

Useful explanation:

- ▶ what changed
- ▶ why it changed
- ▶ how the new version is clearer
- ▶ what you tested after the change
- ▶ what you would improve next

32

Southwest
Tech

Evidence for Week 3



Should show organization and explanation.

Useful evidence includes:

- ▶ working ".py" files
- ▶ function responsibility explanation
- ▶ data structure choice explanation
- ▶ visible output
- ▶ AI-use note if AI was permitted and used

The goal is not just code that runs. The goal is code you can explain.

Photo Credits by iStockphoto. Authorize to be licensed under CC BY-SA.

33

Southwest
Tech

My Mini-App Evidence

Code + Output + Explanation = Understanding

1. My .py Files
These files work together:

- main.py**
Runs the program and shows output
- tasks.py**
Functions for task operations
- data.py**
Holds the task data structure

4. Output Sample
This is what my program prints.

Task List

1. Read Chapter 3 [done]
2. Write summary [in progress]
3. Practice problems [tasks]

2. Function Explanation
Each function has a clear job.

```
get_tasks() Return the list of tasks.
format_tasks() Turn one task into a nicely formatted string.
show_tasks(tasks) Loop through tasks and print each one.
```

3. Data Structure Choice
I chose a list of dictionaries.

```
tasks = [
    {'title': 'Read Chapter 3',
      'status': 'done'},
    {'title': 'Write summary',
      'status': 'in progress'},
    {'title': 'Practice problems',
      'status': 'tasks'}]
```

Why this structure?
Each task has multiple pieces of data, and a list keeps them in order.

What I Learned

- I used functions to organize the work.
- I used a list of dictionaries to store related data.
- I can explain how my program works from data to output.

Evidence Checklist ☒ All files included ☒ Functions explained ☒ Data structure explained ☒ Output shown

Evidence

34

Southwest
Tech

Closing Success Check

If you can justify the improvement, you understand it.
Before submitting Week 3 work, ask:

- ▶ What did I organize?
- ▶ What got clearer?
- ▶ What became easier to change?
- ▶ What evidence shows it still works?
- ▶ Can I explain the final structure?

35

Southwest
Tech

Thank you!

Have Fun!

36
